WCout (C++, Windows)

A simple, consistent output display, dialog utility

Display and format text, numbers, and user-defined types to message boxes, Output to clipboard, memos, or strings using one readable << >> syntax .

```
WCout << "Pi = " << FF-7.2 << PI << SHOW;
WCout << LongMessage >> CLIPBOARD;
```

Quick Start

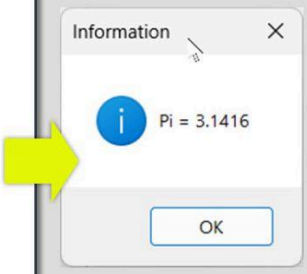
- Add `WCout.cpp` to your project.
- Include `WCout.h` in any file where you want to use it.
- That's it—no external libs or complex setup required.

Every number, text using <<
Every format, control using <<
Every output to Clipboard, Memo >>

Table of Contents

Message Display & Output (SHOW, Clipboard)

Category	Function	Usage
String	Width	WCout << W-30 *W-30 stays active until << W-OFF*
Float	Format	double PI = 3.1416 WCout << FF-7.2 << PI << SHOW; *FF 7.2 sets width=7 decimals=2 (0..9) until FF-OFF*
Message	Display	*WCout << "Pi = " << 3.1416;* WCout << SHOW; WCout << SHOW-Warning; WCout << SHOW-Error; WCout << SHOW-Info; WCout << SHOW-Confirm; WCout << SHOW-Question;
Text	Retrieve	Memo->Lines->Add(WCout.get()); Memo->Lines->Add(WCout()); UnicodeString Text=WCout.get(); WCout >> Text;
	Clipboard	WCout >> CLIPBOARD;
Message	Confirm	bool isConfirm = WCout.DLGConfirm(L"Do you want ketchup with the fries"); *returns true or false*
WCout	Status	WCout << STATUS-ON; WCout << STATUS-OFF; WCout << STATUS-RESET; *Status affects all output globally (handy debugging)* *STATUS-ON (Messages displayed)* *STATUS-OFF (Nothing displayed)* *STATUS-RESET (All settings default)*



Message Formatting Text and Numbers

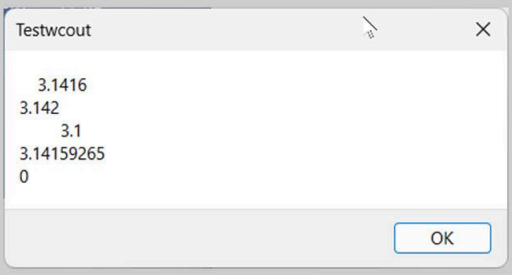
Message

Format data

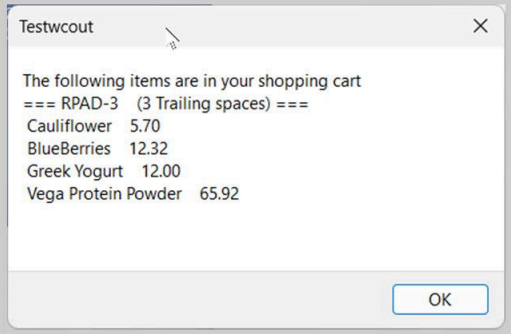
WCout << SHOW;
WCout >> CLIPBOARD;

C++

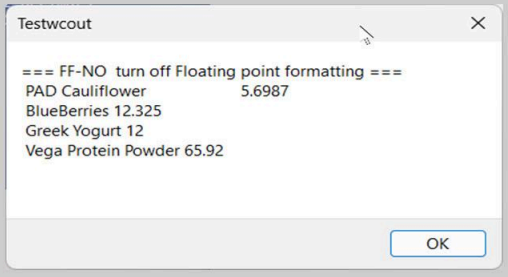
```
void TForm2::SHOWMessage()
{
 double PI = 3.14159265;

      WCout << CLEAR;
      WCout << FF-10.4 << PI << EL  //Float Format 10 wide 4 decimals
       << FF-4.3 << PI << EL
       << FF-12.1 << PI << EL
       << FF-OFF << PI << EL
       << SHOW;
```




```
WCout << L"The following items are in your shopping cart" << EL

      << FF-2.2
      << L"=== RPAD-3    (3 Trailing spaces) ===" << EL
      << RPAD-3   //3 Trailing spaces after text;
      << L" Cauliflower " << 5.6987 << EL
      << L" BlueBerries " << 12.325 << EL
      << L" Greek Yogurt " << 12. << EL
      << L" Vega Protein Powder " << 65.92 << EL << EL
      << RPAD-OFF       //Turn Right Pad OFF
      << SHOW
```

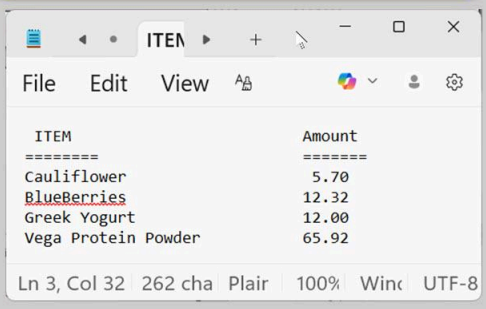



```
WCout << L"=== FF-NO  turn off Floating point formatting ===" << EL
      << FF-OFF //Format Float Spaces OFF
      <<   RPAD-20 <<  L" PAD Cauliflower " << TAB << 5.6987 << EL
      <<  RPAD-OFF
      << L" BlueBerries " << TAB << 12.325 << EL
      << L" Greek Yogurt " << TAB << 12 << EL
      << L" Vega Protein Powder " << 65.92 << EL << EL
      << SHOW
```




```
WCout << L"=== W-30 (String Width 30)   FF-7.2 (Float 7 wide 2 dec.)
===" < EL
      << FF-7.2       //Floating Point Format 7 wide 2 decimals
      << L" W-30 impact more visible in FIXED PITCH FONT " << NL
      << W-30              //String Width = 30
      << L" ITEM "  << L"THE Amount" << NL
      << L"========" << L"=======" << NL
      << L"Cauliflower" << 5.6987 << NL
      << L" BlueBerries"  << 12.325 << NL
      << L" Greek Yogurt" << 12. << NL
      << L" Vega Protein Powder"  << 65.92 << EL
      >> CLIPBOARD;
```

NOTEPAD | NOTEPAD++ | Courier Font
These numbers would line up perfectly with a Fixed-Pitch font sucn as Courier



Simple Confirm Dialog

Category	Function	Usage
Message	Confirm *(Simple)*	bool isConfirm = WCout.DLGConfirm(L"Do you want ketchup with the fries");

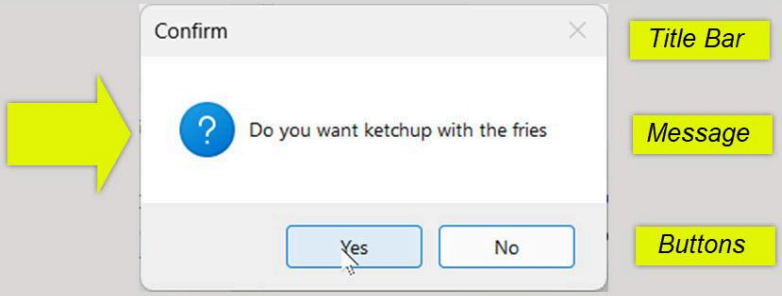
2 *optional* Parameters

```
bool isConfirm = WCout.DLGConfirm(L"Do you want ketchup with the fries");
bool isConfirm = WCout.DLGConfirm(L"Message");  [optional]
bool isConfirm = WCout.DLGConfirm(L"Message", [L"TitleBar"]);
bool isConfirm = WCout.DLGConfirm(L"Message", [btnYES_NO]);  [optional]
bool isConfirm = WCout.DLGConfirm(L"Message", [L"TitleBar"], [btnYES_NO]);
```

DEFAULT TitleBar="Confirm"
DEFAULT buttons=OK/Cancel

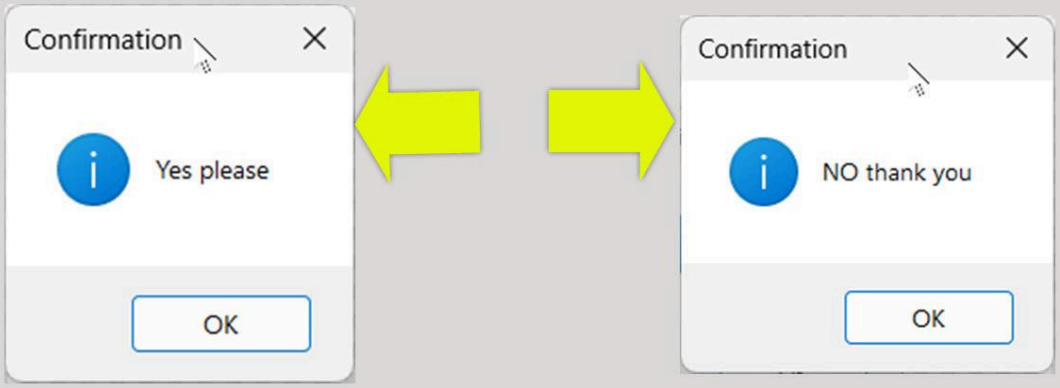
C++

```
void __fastcall TForm2::btnTESTClick(TObject *Sender)
{
  bool isConfirm = WCout.DLGConfirm(L"Do you want ketchup with the fries", btnYES_NO);
. . . . .
```



C++

```
. . . . . .
Txt Msg = isConfirm ? L"Yes please" : L"NO thank you";
WCout << Msg << SHOW-Confirm;
```



Category	Function	Usage
Type	USER Defined	

C++

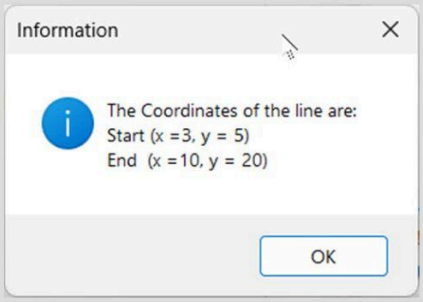
```
struct Point2D {
    int x, y;
};

TWCout& operator<<(TWCout& wc, const Point2D& p) {
    wc << L"(x =" << p.x << L", " << L"y = " << p.y << L")";
    return wc;
}

void __fastcall TForm2::btnTESTMeClick(TObject *Sender)
{

Point2D p1 {3, 5};
Point2D p2 {10, 20};

WCout << L"The Coordinates of the line are: " << FF-7.2 << EL
      << L"Start " << p1 << EL
      << L"End  " << p2 << SHOW-Info;
}
```

The overload is written once - then your type becomes as easy to use as int or String forever

Simplified display

The Magic of operator<< Overloading
*You Write this function **once** - it can be a bit tricky the first time.*
After that?

*Your type works **everywhere** with << - chaining, formatting (FF-, W-), SHOW, clipboard, memos - **zero extra effort.***



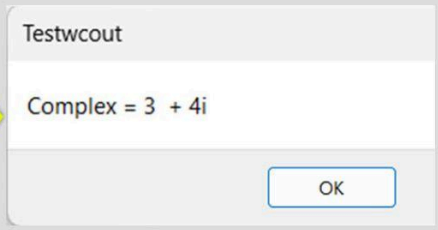
C++

```
class Complex {
public:
  Complex(int Real, int Imaginary) {
      this->Real = Real;
      this->Imaginary = Imaginary;
  }
  int Real;
  int Imaginary;
};
```

Any user type (TWCout&<<UserType) defined)

```
TWCout& operator<<(TWCout& wc, Complex C)
{
  wc << C.Real << L"  + " << C.Imaginary << L"i";
  return wc;
}

void TForm2::ShowComlex() {
   Complex C {3, 4};
   WCout << L"Complex = " << C << SHOW;
}
```

Simplified display



User Defined Type Display

Category	Function	Usage
Type	USER Defined	

C++

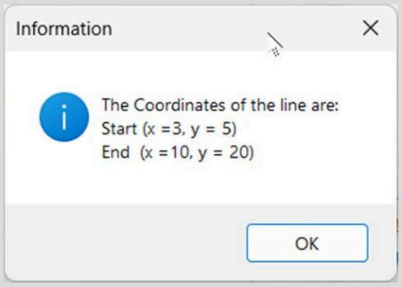
```
struct Point2D {
    int x, y;
};

TWCout& operator<<(TWCout& wc, const Point2D& p) {
    wc << L"(x =" << p.x << L", " << L"y = " << p.y << L")";
    return wc;
}

void __fastcall TForm2::btnTESTMeClick(TObject *Sender)
{

Point2D p1 {3, 5};
Point2D p2 {10, 20};

WCout << L"The Coordinates of the line are: " << FF-7.2 << EL
      << L"Start " << p1 << EL
      << L"End  " << p2 << SHOW-Info;
}
```

The overload is written once - then your type becomes as easy to use as int or String forever

Simplified display

The Magic of operator<< Overloading
*You Write this function **once** - it can be a bit tricky the first time.*
After that?

*Your type works **everywhere** with << - chaining, formatting (FF-, W-), SHOW, clipboard, memos - **zero extra effort.***



C++

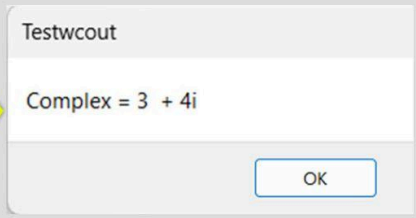
```
class Complex {
public:
  Complex(int Real, int Imaginary) {
      this->Real = Real;
      this->Imaginary = Imaginary;
  }
  int Real;
  int Imaginary;
};
```

Any user type (TWCout&<<UserType) defined)

TWCout& operator<<(TWCout& wc, Complex C)

```
{
  wc << C.Real << L"  + " << C.Imaginary << L"i";
  return wc;
}

void TForm2::ShowComlex() {
   Complex C {3, 4};
   WCout << L"Complex = " << C << SHOW;
}
```

Simplified display



Spaces in large blocks of text (touching text/data)